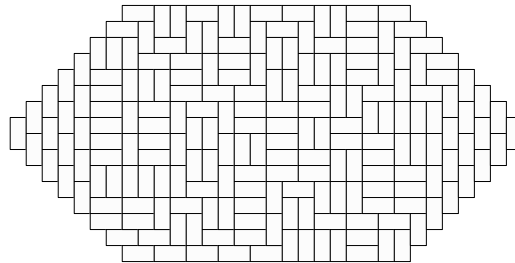
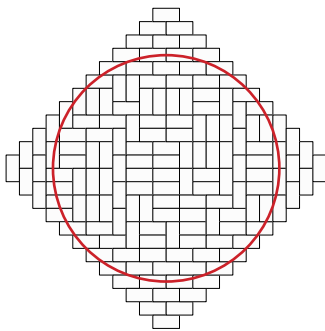


September 8, 2026 at 19:35

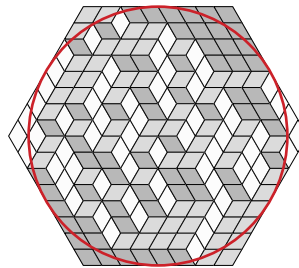
1. Introduction. Exercise 7.2.2.1–262 asks how big a ZDD Algorithm Z builds for two families of tilings. Part (a) takes S_n , a $16 \times n$ rectangle with a right triangle of side 7 cut from each corner, and asks for the number of ZDD nodes, and for the number of domino tilings of S_{16} and of S_{32} . Part (b) takes T_n , a hexagon of sides $(8, 8, n, 8, 8, n)$, and asks for the size of the diagram that holds all of its diamond tilings.



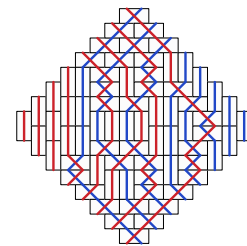
S_{32}



the arctic circle



T_{888}



Randall's paths

The answer gives two linear formulas, two long exact counts, and a page of remarks around them: which way the cells must be numbered, whether the MRV heuristic helps, how many diamonds of each kind a tiling has, an arctic circle, and a picture of blue and red paths that D. Randall found hiding inside a domino tiling. This program checks all of it.

Everything came out. The one thing that did not is in the answer's recipe for part (b): as printed, the items it names describe a region with more upward triangles than downward ones, and the resulting problem has no tilings at all. One bound is missing, and §26 says which.

```
package main
import (
    "flag"
    "fmt"
    "math"
    "math/big"
    "math/rand/v2"
    "sort"
    "strconv"
    "strings"
    "time"
    zdd "github.com/sjnam/dancing-cells/zdd"
)
```

```

⟨ Types 5 ⟩
⟨ Functions 4 ⟩
func main() {
    ⟨ Read the command line 2 ⟩
    ⟨ Do what the mode asks 3 ⟩
}

```

2. The modes run in the order the answer reads. Most of them take a minute or two together; **nodes** alone builds eighteen diagrams, the largest with more than two million nodes.

⟨ Read the command line 2 ⟩ $\equiv$

```

mode := flag.String("mode", "all",
    "aztec, nodes, order, mrv, hex, rows, paths, arctic, or all")
flag.Parse()
all := *mode  $\equiv$  "all"

```

This code is used in section 1.

3. ⟨ Do what the mode asks 3 ⟩ $\equiv$

```

if *mode  $\equiv$  "aztec"  $\vee$  all {
    ⟨ Count the tilings of an Aztec diamond 20 ⟩
}
if *mode  $\equiv$  "nodes"  $\vee$  all {
    ⟨ Measure the diagram of  $S_n$  22 ⟩
}
if *mode  $\equiv$  "order"  $\vee$  all {
    ⟨ Number the cells the other way 24 ⟩
}
if *mode  $\equiv$  "mrv"  $\vee$  all {
    ⟨ Weigh the two branching rules 25 ⟩
}
if *mode  $\equiv$  "hex"  $\vee$  all {
    ⟨ Measure the diagram of  $T_n$  26 ⟩
}
if *mode  $\equiv$  "rows"  $\vee$  all {
    ⟨ Count the diamonds of each kind 31 ⟩
}
if *mode  $\equiv$  "paths"  $\vee$  all {
    ⟨ Trace Randall's paths 38 ⟩
}
if *mode  $\equiv$  "arctic"  $\vee$  all {
    ⟨ Look for the arctic circle 43 ⟩
}

```

This code is used in section 1.

4. The shape S_{mn} . The exercise fixes the height at 16 and cuts triangles of side 7. Its answer then loosens both at once—“the more general shapes S_{mn} considered here, where we replace 16 by $2m$ and 7 by $m - 1$ ”—and that is the shape this program builds, because S_n is S_{8n} and the Aztec diamond of order m is $S_{m(2m)}$. A cell survives when it is far enough from all four cut corners.

⟨ Functions 4 ⟩ $\equiv$

```
func inS(r, c, m, n int) bool {
  return  $r + c \geq m - 1 \wedge r + (n - 1 - c) \geq m - 1 \wedge$ 
     $(2 * m - 1 - r) + c \geq m - 1 \wedge (2 * m - 1 - r) + (n - 1 - c) \geq m - 1$ 
}
```

This code is also defined in sections 6, 10, 12, 18, 19, 28, 32, 36, 37.

This code is used in section 1.

5. A board is the whole exact cover problem: the surviving cells in the order they will be named, the dominoes in the order they will be numbered, and the DLX text that says so. Element k of the ZDD is $opts[k]$, which is how every later section reads a tiling back.

⟨ Types 5 ⟩ $\equiv$

```
type domino struct {
  p, q int // item numbers,  $p < q$ 
  horiz bool
}
type board struct {
  m, n int
  cells [[2]int]
  num map[[2]int]int
  opts []domino
  text string
}
```

This code is also defined in sections 11, 17.

This code is used in section 1.

6. Answer 262 says that the order of the cells is critical: numbering them columnwise, top to bottom and then left to right, keeps the diagram small, while numbering them rowwise blows it up. Both are wanted, so *rowwise* picks between them.

⟨ Functions 4 ⟩ $+ \equiv$

```
func newBoard(m, n int, rowwise bool) *board {
  b := &board{m: m, n: n, num: map[[2]int]int{}}
  ⟨ Number the cells 7 ⟩
  ⟨ Make the dominoes 8 ⟩
  ⟨ Write the DLX text 9 ⟩
  return b
}
```

7. Counting i through all $2mn$ places of the frame visits every cell of the frame once in either numbering, so one loop serves both.

⟨Number the cells 7⟩ $\equiv$

```

for  $i := 0; i < 2 * m * n; i++$  {
   $r, c := i / n, i \% n$ 
  if  $\neg \text{rowwise}$  {
     $r, c = i \% (2 * m), i / (2 * m)$ 
  }
  if  $\text{inS}(r, c, m, n)$  {
     $b.\text{num}[[2]\text{int}\{r, c\}] = \text{len}(b.\text{cells})$ 
     $b.\text{cells} = \text{append}(b.\text{cells}, [2]\text{int}\{r, c\})$ 
  }
}

```

This code is used in section 6.

8. Each cell offers the domino running right from it and the domino running down; in either numbering the cell it is offered from is the lower-numbered of the two, so the options come out grouped by their smallest item, which is what makes the diagram an ordered one.

⟨Make the dominoes 8⟩ $\equiv$

```

for  $-, p := \text{range } b.\text{cells}$  {
  for  $k, q := \text{range } [[2]\text{int}\{\{p[0], p[1] + 1\}, \{p[0] + 1, p[1]\}\}]$  {
     $i, ok1 := b.\text{num}[p]$ 
     $j, ok2 := b.\text{num}[q]$ 
    if  $\neg ok1 \vee \neg ok2$  {
      continue
    }
    if  $i > j$  {
       $i, j = j, i$ 
    }
     $b.\text{opts} = \text{append}(b.\text{opts}, \text{domino}\{i, j, k \equiv 0\})$ 
  }
}

```

This code is used in section 6.

9. ⟨Write the DLX text 9⟩ $\equiv$

```

var  $sb$   $\text{strings.Builder}$ 
for  $i := \text{range } b.\text{cells}$  {
  if  $i > 0$  {
     $sb.\text{WriteByte}(' \sqcup')$ 
  }
   $\text{fmt.Fprintf}(\&sb, "c\%d", i)$ 
}
 $sb.\text{WriteByte}('\n')$ 
for  $-, o := \text{range } b.\text{opts}$  {
   $\text{fmt.Fprintf}(\&sb, "c\%d\sqcup c\%d\n", o.p, o.q)$ 
}
 $b.\text{text} = sb.\text{String}()$ 

```

This code is used in section 6.

10. The hexagon T_{lmn} . Part (b) works in the triangle coordinates of answer 124: a square cell (x, y) , sheared and scaled, becomes an upward triangle (x, y) together with the downward triangle $(x, y)'$ immediately to its right. A diamond covers one of each, so the options of answer 262 pair (x, y) with $(x, y)'$, with $(x, y - 1)'$, or with $(x - 1, y)'$ —leaning one way, standing upright, leaning the other.

The hexagon of sides (l, m, n, l, m, n) is the frame $0 \leq x < n + m$, $0 \leq y < l + m$ with two opposite corners cut off. Answer 262 names the cut for the downward triangles in full; for the upward ones it names only the lower bound, and *printed* is here so that §26 can show what that leaves.

⟨Functions 4⟩ +≡

```

func upTri( $x, y, l, m, n$  int, printed bool) bool {
  if  $\neg(0 \leq x \wedge x < n + m \wedge 0 \leq y \wedge y < l + m \wedge x + y \geq m)$  {
    return false
  }
  return printed  $\vee x + y < n + l + m$ 
}

func dnTri( $x, y, l, m, n$  int) bool {
  return  $0 \leq x \wedge x < n + m \wedge 0 \leq y \wedge y < l + m \wedge$ 
     $m - 1 \leq x + y \wedge x + y < n + l + m - 1$ 
}

```

11. ⟨Types 5⟩ +≡

```

type tri struct {
   $x, y$  int
  down bool
}

type diamond struct {
   $p, q$  int    // item numbers,  $p < q$ 
   $x, y$  int    // the upward triangle it covers
  kind int    // 0, 1, 2 for  $(x, y)'$ ,  $(x, y - 1)'$ ,  $(x - 1, y)'$ 
}

type hexagon struct {
   $l, m, n$  int
  tris []tri
  num map[tri]int
  opts []diamond
  text string
}

```

12. The triangles are numbered by x and then by y , the upward one first. That is the order answer 262 lists them in, and §30 shows what it is worth; passing *across* numbers them by y and then x instead, for the comparison there.

⟨Functions 4⟩ +≡

```

func newHexagon( $l, m, n$  int, printed, across bool) *hexagon {
   $h := \&\mathbf{hexagon}\{l: l, m: m, n: n, num: \mathbf{map}[\mathbf{tri}]\mathbf{int}\{\}\}$ 
  ⟨Number the triangles 13⟩
  ⟨Make the diamonds 14⟩
  ⟨Write the hexagon's DLX text 16⟩
  return  $h$ 
}

```

13. $\langle \text{Number the triangles } 13 \rangle \equiv$

```

for  $i := 0; i < (n + m) * (l + m); i++$  {
   $x, y := i / (l + m), i \% (l + m)$ 
  if across {
     $x, y = i \% (n + m), i / (n + m)$ 
  }
  for  $\_, down := \text{range } []\text{bool } \{ \text{false}, \text{true} \}$  {
    if  $down \wedge \neg dnTri(x, y, l, m, n)$  {
      continue
    }
    if  $\neg down \wedge \neg upTri(x, y, l, m, n, printed)$  {
      continue
    }
     $t := \text{tri}\{x, y, down\}$ 
     $h.num[t] = len(h.tris)$ 
     $h.tris = append(h.tris, t)$ 
  }
}

```

This code is used in section 12.

14. Here the lower-numbered item of a diamond is not always the upward triangle, so the options have to be sorted into buckets by their smallest item before they are written out.

$\langle \text{Make the diamonds } 14 \rangle \equiv$

```

 $buckets := make([], [], \text{diamond}, len(h.tris))$ 
for  $\_, t := \text{range } h.tris$  {
  if  $t.down$  {
    continue
  }
   $\langle \text{Offer the three diamonds that cover } (x, y) \text{ } 15 \rangle$ 
}
for  $\_, bucket := \text{range } buckets$  {
   $h.opts = append(h.opts, bucket...)$ 
}

```

This code is used in section 12.

15. $\langle \text{Offer the three diamonds that cover } (x, y) \text{ } 15 \rangle \equiv$

```

for  $k, d := \text{range } [[2]\text{int } \{ \{t.x, t.y\}, \{t.x, t.y - 1\}, \{t.x - 1, t.y\} \} \{$ 
  if  $\neg dnTri(d[0], d[1], l, m, n)$  {
    continue
  }
   $i, j := h.num[t], h.num[\text{tri}\{d[0], d[1], \text{true}\}]$ 
  if  $i > j$  {
     $i, j = j, i$ 
  }
   $buckets[i] = append(buckets[i], \text{diamond}\{i, j, t.x, t.y, k\})$ 
}

```

This code is used in section 14.

16. $\langle$ Write the hexagon's DLX text 16 $\rangle \equiv$

```

var sb strings.Builder
for i := range h.tris {
    if i > 0 {
        sb.WriteByte('␣')
    }
    fmt.Fprintf(&sb, "%d", i)
}
sb.WriteByte('\n')
for _, o := range h.opts {
    fmt.Fprintf(&sb, "%d␣t%d\n", o.p, o.q)
}
h.text = sb.String()

```

This code is used in section 12.

17. Running the engine. One call builds a diagram and reports what it cost. The engine is the *zdd* package of this repository, which is Algorithm Z on sparse sets; *mrsv* chooses between its two branching rules, and everything else is the same.

⟨Types 5⟩ +≡

```
type result struct {
    d *zdd.Diagram
    sv *zdd.Solver
    secs time.Duration
}
```

18. ⟨Functions 4⟩ +≡

```
func solve(text string, mrsv bool) result {
    t := time.Now()
    sv := zdd.New()
    sv.MRV = mrsv
    d := sv.Dance(strings.NewReader(text))
    return result{d, sv, time.Since(t)}
}
```

19. Two small conveniences. *bigStr* makes a decimal string of a power of two, to compare against a count; *formula* is the linear function the answer predicts, with a word for whether it holds.

⟨Functions 4⟩ +≡

```
func pow2(k int) *big.Int {
    return new(big.Int).Lsh(big.NewInt(1), uint(k))
}

func agrees(got, want int) string {
    if got ≡ want {
        return "yes"
    }
    return fmt.Sprintf("no, off by %d", got - want)
}
```


20. Aztec diamonds. The exercise calls S_{16} “the Aztec diamond of order 8,” and the answer adds that an Aztec diamond of order m has exactly $2^{m(m+1)/2}$ domino tilings. Both are easy to see here, because $S_{m(2m)}$ is the Aztec diamond of order m : its row r holds $2 \min(r+1, 2m-r)$ cells, so the rows run $2, 4, \dots, 2m, 2m, \dots, 4, 2$.

⟨ Count the tilings of an Aztec diamond 20 ⟩ $\equiv$

```
fmt.Println("Aztec_diamonds")
for m := 1; m ≤ 10; m++ {
  ⟨ Check that row r of  $S_{m(2m)}$  has  $2 \min(r+1, 2m-r)$  cells 21 ⟩
  r := solve(newBoard(m, 2 * m, false).text, false)
  fmt.Printf("order %2d: %26s_tilings, %2^-3d %v, %8d_nodes\n",
    m, r.d.Count(), m * (m + 1) / 2, r.d.Count().Cmp(pow2(m * (m + 1) / 2)) ≡ 0,
    r.d.Nodes())
}
```

This code is used in section 3.

21. ⟨ Check that row r of $S_{m(2m)}$ has $2 \min(r+1, 2m-r)$ cells 21 ⟩ $\equiv$

```
for row := 0; row < 2 * m; row++ {
  w := 0
  for c := 0; c < 2 * m; c++ {
    if inS(row, c, m, 2 * m) {
      w++
    }
  }
  if w ≠ 2 * min(row + 1, 2 * m - row) {
    fmt.Printf("order %d, row %d: %d cells, not a diamond!\n", m, row, w)
  }
}
```

This code is used in section 20.

22. The size of the diagram. Now for part (a) proper. Answer 262 says that columnwise numbering “yields *linear* ZDD size,” and that the number of nodes is $154440n - 2655855$ for all $n \geq 30$.

There is a question of what we are entitled to compare. Algorithm Z outputs a *free* ZDD, whose shape depends on the order step Z3 branches in; but exercise 264 shows that branching on the least-numbered item makes the output *ordered*, and exercise 265 that no two of its nodes are ever equal, so in that case the output is the reduced ordered ZDD for the option numbering—the one canonical object, which is exactly what the *bdd* library builds. So the counts are comparable, and §25 shows the other side of the same coin: our number cannot depend on the branching rule at all.

⟨ Measure the diagram of S_n 22 ⟩ $\equiv$

```
fmt.Println("The diagram of S_n, cells numbered columnwise")
fmt.Println("the formula is claimed only for n >= 30")
for n := 15; n ≤ 32; n++ {
  r := solve(newBoard(8, n, false).text, false)
  want := 154440 * n - 2655855
  fmt.Printf("n=%2d: %39s tilings, %7d nodes, %154440n-2655855: %s\n",
    n, r.d.Count(), r.d.Nodes(), agrees(r.d.Nodes(), want))
  ⟨ Check the two counts the exercise asks for 23 ⟩
}
```

This code is used in section 3.

23. “There are $68719476736 = (\sqrt{2})^{72}$ solutions for S_{16} , via exercise 7.1.4–208; for S_{32} there are $152326556015596771390830202722034115329 \approx 1.552^{200}$.”

⟨ Check the two counts the exercise asks for 23 ⟩ $\equiv$

```
switch n {
case 16:
  fmt.Printf("S_16 = 2^36 = 68719476736: %v\n",
    r.d.Count().String() ≡ "68719476736")
case 32:
  const s32 = "152326556015596771390830202722034115329"
  f, _ := new(big.Float).SetInt(r.d.Count()).Float64()
  fmt.Printf("S_32 = %s: %v, and it is %.4f^200\n",
    s32, r.d.Count().String() ≡ s32, math.Pow(f, 1.0/200))
}
```

This code is used in section 22.

24. Which way to number the cells. “Rowwise ordering (left-to-right, top-to-bottom) causes exponential growth.” It does: the diagram grows by a factor of about 1.967 for every column added, where columnwise numbering adds a constant 154440.

⟨Number the cells the other way 24⟩ $\equiv$

```

fmt.Println("The same problem, cells numbered rowwise")
prev := 0
for n := 16; n ≤ 22; n += 2 {
  r := solve(newBoard(8, n, false).text, false)
  q := solve(newBoard(8, n, true).text, false)
  growth := "--"
  if prev > 0 {
    growth = fmt.Sprintf("%.3f",
      math.Sqrt(float64(q.d.Nodes())/float64(prev)))
  }
  prev = q.d.Nodes()
  fmt.Printf("n=%2d: columnwise %7d nodes, rowwise %8d, per column %s\n",
    n, r.d.Nodes(), q.d.Nodes(), growth)
}

```

This code is used in section 3.

25. Fewest options, or least numbered. “It turns out to be better *not* to use the MRV heuristic, when $n \geq 18$.” The engine here has both rules, and the crossing is exactly where the answer puts it: at $n = 17$ fewest-options needs fewer memo entries, at $n = 18$ it needs more, and it stays behind from there on.

The diagram itself never moves. It cannot: a reduced ordered ZDD is determined by the family and the variable order, and the branching rule is neither.

⟨ Weigh the two branching rules 25 ⟩ $\equiv$

```

fmt.Println("Fewest_options_(MRV)_against_least_numbered")
for n := 15; n ≤ 22; n++ {
    text := newBoard(8, n, false).text
    a, b := solve(text, true), solve(text, false)
    better := "least_numbered"
    if a.sv.Signatures() < b.sv.Signatures() {
        better = "MRV"
    }
    fmt.Printf("n=%2d: %8d memos against %8d, %-14s; same diagram: %v\n",
        n, a.sv.Signatures(), b.sv.Signatures(), better,
        a.d.Nodes() ≡ b.d.Nodes())
}

```

This code is used in section 3.

26. The hexagon's items. Part (b) of answer 262 sets the problem up like this:

“use items (x, y) for $0 \leq x < n + 8$, $0 \leq y < 16$, $x + y \geq 8$; $(x, y)'$ for $0 \leq x < n + 8$, $0 \leq y < 16$, $7 \leq x + y < n + 15$.”

The second clause cuts two corners off the frame; the first cuts only one. That leaves $16(n + 8) - 36$ upward triangles against $16(n + 8) - 73$ downward ones, and a region with unequal numbers of the two cannot be tiled by diamonds at all. The missing bound is $x + y < n + 16$: with it both counts come to $16n + 64 = lm + mn + nl$ for $l = m = 8$, which is what a hexagon of sides $(8, 8, n, 8, 8, n)$ must have.

⟨Measure the diagram of T_n 26⟩ $\equiv$

```
fmt.Println("The items of T_n, as printed and as repaired")
for _, printed := range []bool { true, false } {
    h := newHexagon(8, 8, 6, printed, false)
    up, down := 0, 0
    for _, t := range h.tris {
        if t.down {
            down++
        } else {
            up++
        }
    }
    fmt.Printf("n=%d, s=%d up, %d down, s tilings\n",
        n, up, down, map[bool]string{true: "printed:", false: "repaired:"}[printed],
        up, down, solve(h.text, false).d.Count())
}
```

This code is also defined in sections 27, 29, 30.

This code is used in section 3.

27. “Then the ZDD size (without MRV) turns out to be $257400n - 1210061$, for all $n \geq 7$.” It does, from $n = 7$ on and not before; at $n = 6$ the diagram has 350784 nodes where the formula asks for 334339.

The count itself is MacMahon’s: a diamond tiling of the hexagon T_{lmn} is a plane partition in an $l \times m \times n$ box, and there are $\Pi_{lmn} = \prod \prod \prod (i + j + k - 1) / (i + j + k - 2)$ of them.

⟨Measure the diagram of T_n 26⟩ $+ \equiv$

```
fmt.Println("The diagram of T_n")
for n := 4; n <= 10; n++ {
    h := newHexagon(8, 8, n, false, false)
    r := solve(h.text, false)
    fmt.Printf("n=%2d: %28s tilings, MacMahon %v, %7d nodes, %s\n",
        n, r.d.Count(), r.d.Count().Cmp(macMahon(8, 8, n)) == 0,
        r.d.Nodes(), agrees(r.d.Nodes(), 257400 * n - 1210061))
}
```

28. $\langle \text{Functions 4} \rangle + \equiv$

```

func macMahon(l, m, n int) *big.Int {
    num, den := big.NewInt(1), big.NewInt(1)
    for i := 1; i ≤ l; i++ {
        for j := 1; j ≤ m; j++ {
            for k := 1; k ≤ n; k++ {
                num.Mul(num, big.NewInt(int64(i + j + k - 1)))
                den.Mul(den, big.NewInt(int64(i + j + k - 2)))
            }
        }
    }
    return num.Quo(num, den)
}

```

29. Answer 262 names two of those products, Π_{888} and $\Pi_{88(16)}$, and this is the place to check them and the formula on the largest case the answer mentions.

$\langle \text{Measure the diagram of } T_n \text{ 26} \rangle + \equiv$

```

for n, n := range []int { 8, 16 } {
    r := solve(newHexagon(8, 8, n, false, false).text, false)
    fmt.Printf("Pi_88(%d) = %s: %v, %7d nodes, %s\n", n, r.d.Count(),
        r.d.Count().Cmp(macMahon(8, 8, n)) ≡ 0, r.d.Nodes(),
        agrees(r.d.Nodes(), 257400 * n - 1210061))
}

```

30. The order of the triangles matters here as much as the order of the cells did in part (a). Numbering by y and then by x —across the hexagon rather than along it—starts out smaller, but it is not linear: it roughly quadruples where the answer's numbering adds a constant, and it has overtaken by $n = 9$.

$\langle \text{Measure the diagram of } T_n \text{ 26} \rangle + \equiv$

```

fmt.Println("and with the triangles numbered across instead of along:")
for n := 3; n ≤ 9; n += 2 {
    along := newHexagon(8, 8, n, false, false)
    across := newHexagon(8, 8, n, false, true)
    fmt.Printf("n=%d: %7d nodes against %7d\n", n,
        solve(across.text, false).d.Nodes(),
        solve(along.text, false).d.Nodes())
}

```

31. Diamonds by kind, and by row. The answer then reads a tiling of the hexagon as a plane partition, and says what that forces: “every tiling of T_n has respectively $(1, 2, \dots, 8, 7, \dots, 1)$ vertical diamonds in rows $(1, 2, \dots, 15)$, hence 64 in all; and these occurrences are nested.”

The three kinds of diamond are the three ways to lean, and the plane-partition picture says that a tiling of T_{lmn} has lm , mn and nl of them—so for $l = m = 8$ the upright ones, kind 1, always number 64. Every one of these is a statement about *all* tilings at once, which is where a diagram earns its keep: *MaxWeight* finds the largest number of kind-1 diamonds any tiling has, and the same call on negated weights finds the smallest, both in one walk over the DAG. When the two agree the count is forced.

⟨ Count the diamonds of each kind 31 ⟩ $\equiv$

```
fmt.Println("Diamonds by kind")
for _, s := range [][]int {{8, 8, 2}, {3, 4, 5}, {8, 8, 8}} {
    l, m, n := s[0], s[1], s[2]
    h := newHexagon(l, m, n, false, false)
    d := solve(h.text, false).d
    fmt.Printf("T_%d,%d,%d:kinds", l, m, n)
    for kind := 0; kind < 3; kind++ {
        lo, hi := span(d, h, func(o diamond) bool { return o.kind == kind })
        fmt.Printf("%s", spanStr(lo, hi))
    }
    fmt.Printf(" (%nl, %lm, %mn) = (%d, %d, %d)\n", n * l, l * m, m * n)
}
```

This code is also defined in sections 33, 34.

This code is used in section 3.

32. ⟨ Functions 4 ⟩ $+ \equiv$

```
func span(d *zdd.Diagram, h *hexagon, pick func(diamond) bool)(int, int) {
    w := make([]int, len(h.opts) + 1)
    for k, o := range h.opts {
        if pick(o) {
            w[k + 1] = 1
        }
    }
    _, hi, _ := d.MaxWeight(w)
    for i := range w {
        w[i] = -w[i]
    }
    _, lo, _ := d.MaxWeight(w)
    return -lo, hi
}

func spanStr(lo, hi int) string {
    if lo == hi {
        return strconv.Itoa(lo)
    }
    return fmt.Sprintf("%d..%d", lo, hi)
}
```

33. Row by row now. A kind-1 diamond stands on the boundary between rows $y - 1$ and y of the frame, so its row is y , and y runs from 1 to $l + m - 1$: for $l = m = 8$ that is the fifteen rows the answer names.

⟨ Count the diamonds of each kind 31 ⟩ +≡

```
fmt.Println("Upright_diamonds_row_by_row,T_8,8,n")
for _, n := range []int { 2, 5, 8 } {
    h := newHexagon(8, 8, n, false, false)
    d := solve(h.text, false).d
    fmt.Printf("n=%d:", n)
    for y := 1; y < 16; y++ {
        lo, hi := span(d, h, func(o diamond) bool {
            return o.kind == 1 & o.y == y
        })
        fmt.Printf("%s", spanStr(lo, hi))
    }
    fmt.Println()
}
```

34. “And these occurrences are nested.” They interlace: an upright diamond of row y sits at $2x + y$ along that row, and the positions in one row fall strictly between the positions in the next, the longer row surrounding the shorter. That is precisely the interlacing that turns a tiling into a plane partition. Here it is checked on every tiling of two small hexagons, and on a thousand random tilings of $T_{8,8,4}$ —random, of course, being another thing the diagram gives for nothing.

⟨ Count the diamonds of each kind 31 ⟩ +≡

```
fmt.Println("Interlacing")
for _, s := range [][]int { {2, 2, 2}, {3, 3, 2} } {
    h := newHexagon(s[0], s[1], s[2], false, false)
    d := solve(h.text, false).d
    z, root := d.ZDD()
    seen, bad := 0, 0
    for elts := range z.Subsets(root) {
        seen++
        if !interlaced(h, elts) {
            bad++
        }
    }
    fmt.Printf("T_%d,%d,%d: all %d tilings, %d not interlaced\n",
        s[0], s[1], s[2], seen, bad)
}
```

⟨ Sample $T_{8,8,4}$ at random and check the interlacing 35 ⟩

35. $\langle \text{Sample } T_{8,8,4} \text{ at random and check the interlacing 35} \rangle \equiv$

```

h := newHexagon(8,8,4,false,false)
d := solve(h.text,false).d
z,root := d.ZDD()
rnd := rand.New(rand.NewPCG(20260908,262))
bad := 0
for i := 0; i < 1000; i++ {
    elts,_ := z.Random(root,rnd)
    if ¬interlaced(h,elts) {
        bad++
    }
}
fmt.Printf("T_8,8,4:1000randomtilings,%dnotinterlaced\n",bad)

```

This code is used in section 34.

36. $\langle \text{Functions 4} \rangle + \equiv$

```

func interlaced(h *hexagon,elts []int) bool {
    rows := make([][]int,h.l+h.m)
    for _,k := range elts {
        if o := h.opts[k]; o.kind == 1 {
            rows[o.y] = append(rows[o.y],2*o.x+o.y)
        }
    }
    for _,v := range rows {
        sort.Ints(v)
    }
    for y := 1; y+1 < len(rows); y++ {
        if ¬strictlyBetween(rows[y],rows[y+1]) {
            return false
        }
    }
    return true
}

```

37. The shorter list must have one entry fewer than the longer, and its entries must fall in the gaps.

$\langle \text{Functions 4} \rangle + \equiv$

```

func strictlyBetween(a,b []int) bool {
    if len(a) > len(b) {
        a,b = b,a
    }
    if len(b) ≠ len(a)+1 {
        return false
    }
    for i,v := range a {
        if ¬(b[i] < v ∧ v < b[i+1]) {
            return false
        }
    }
    return true
}

```

38. Randall's paths. Answer 262 ends part (a) with a picture and a caption: “every vertical domino has either a blue or red path; every horizontal domino has blue and red paths, crossed.” That is enough to draw them. Colour the cells like a chessboard and put a node at the midpoint of every horizontal edge of the grid. Then

- a vertical domino carries one straight segment, from the node above it to the node below it, and
- a horizontal domino carries two, each running from the top of one of its cells to the bottom of the other—blue from the black cell down to the white, red from the white down to the black, so that they cross.

Nothing else is free, and the drawing that results is the one on page 483. What has to be checked is that the segments join up: that no node is left with two segments coming in or two going out, that a segment never joins a blue node to a red one, and that every path so formed runs from the top of the region to the bottom. They do, and there are m paths of each colour in an Aztec diamond of order m .

⟨ Trace Randall's paths 38 ⟩ $\equiv$

```
fmt.Println("Randall's blue and red paths")
for _, m := range []int { 4, 6, 8 } {
    b := newBoard(m, 2 * m, false)
    d := solve(b.text, false).d
    z, root := d.ZDD()
    rnd := rand.New(rand.NewPCG(20260908, uint64(m)))
    bad, blue, red := 0, 0, 0
    for i := 0; i < 200; i++ {
        elts, _ := z.Random(root, rnd)
        ⟨ Lay the segments of one tiling 39 ⟩
        ⟨ Follow the segments and count the paths 42 ⟩
    }
    fmt.Printf("order%2d: %20d tilings, %d faulty; %d blue and %d red" +
        " paths each time\n", m, bad, blue/200, red/200)
}
```

This code is used in section 3.

39. A node is written as the pair (row, column) of the cell below it, so the node above cell (r, c) is (r, c) and the node below it is $(r + 1, c)$, and the node is blue when $r + c$ is even. Since the cells are numbered columnwise, $o.p$ is always the left or upper cell of its domino.

⟨ Lay the segments of one tiling 39 ⟩ $\equiv$

```
next, prev := map[[2]int][2]int {}, map[[2]int]bool {}
ok := true
for _, k := range elts {
    o := b.opts[k]
    p := b.cells[o.p]
    segs := [[2]int]{ { {p[0], p[1]}, {p[0] + 2, p[1]} } }
    if o.horiz {
        ⟨ Cross a blue segment and a red one over the domino 40 ⟩
    }
    ⟨ Add the segments, watching for a clash 41 ⟩
}
```

This code is used in section 38.

40. $\langle$ Cross a blue segment and a red one over the domino 40 $\rangle \equiv$

```

black, white := p[1], p[1] + 1
if (p[0] + p[1]) % 2 ≠ 0 {
    black, white = white, black
}
segs = [][2][2]int{
    {{p[0], black}, {p[0] + 1, white}},
    {{p[0], white}, {p[0] + 1, black}},
}

```

This code is used in section 39.

41. $\langle$ Add the segments, watching for a clash 41 $\rangle \equiv$

```

for _, s := range segs {
    _, twice := next[s[0]]
    if twice ∨ prev[s[1]] ∨ (s[0][0] + s[0][1]) % 2 ≠ (s[1][0] + s[1][1]) % 2 {
        ok = false
    }
    next[s[0]] = s[1]
    prev[s[1]] = true
}

```

This code is used in section 39.

42. A path starts at a node with nothing coming in. Following it must leave the region at the bottom, never at the side and never at the top, and the parity of the node it started from says which colour it is.

⟨ Follow the segments and count the paths 42 ⟩ $\equiv$

```

for  $s := \text{range } next$  {
  if  $prev[s]$  {
    continue
  }
  if  $\neg, inside := b.num[[2]int\{s[0] - 1, s[1]\}]; inside$  {
     $ok = \text{false}$ 
  }
   $e := s$ 
  for {
     $n, more := next[e]$ 
    if  $\neg more$  {
      break
    }
     $e = n$ 
  }
  if  $\neg, inside := b.num[e]; inside$  {
     $ok = \text{false}$ 
  }
  if  $(s[0] + s[1]) \% 2 \equiv 0$  {
     $blue++$ 
  } else {
     $red++$ 
  }
}
if  $\neg ok$  {
   $bad++$ 
}

```

This code is used in section 38.

43. The arctic circle. “As $m \rightarrow \infty$, the dominoes at the corners are q.s. aligned, except within an ‘arctic circle’ of radius $m/\sqrt{2}$.” Sampling would show it, but the diagram can do better than sample: for each cell it can say *exactly* how likely that cell is to be covered lengthwise, because the tilings that use a given domino are the quotient of the family by that element, and counting them is one more walk over the DAG.

Write p for that chance and $2p - 1$ for how far the cell is from undecided. Averaged over the cells at each radius, it stays near zero out to $0.7m$ and then climbs to one—the circle of radius $m/\sqrt{2} = 0.7071m$ falling right in the middle of the climb, and the climb getting steeper as m grows.

⟨ Look for the arctic circle 43 ⟩ $\equiv$

```
fmt.Println("The arctic circle: mean | 2p-1 | by radius, in tenths of m")
for _, m := range []int { 4, 6, 8, 10 } {
    b := newBoard(m, 2 * m, false)
    d := solve(b.text, false).d
    ⟨ Find the chance that each cell is covered lengthwise 44 ⟩
    ⟨ Report how far from undecided the cells are, band by band 45 ⟩
}
```

This code is used in section 3.

44. ⟨ Find the chance that each cell is covered lengthwise 44 ⟩ $\equiv$

```
z, root := d.ZDD()
total := new(big.Float).SetInt(d.Count())
chance := make([]float64, len(b.cells))
for k, o := range b.opts {
    if ¬o.horiz {
        continue
    }
    share := new(big.Float).SetInt(z.Count(z.Quotient(root, z.Elt(k))))
    q, _ := share.Quo(share, total).Float64()
    chance[o.p] += q
    chance[o.q] += q
}
```

This code is used in section 43.

45. $\langle$ Report how far from undecided the cells are, band by band 45 $\rangle \equiv$

```

const bands = 10
var sum [bands]float64
var cnt [bands]int
for i, p := range b.cells {
    y, x := float64(p[0]) + .5 - float64(m), float64(p[1]) + .5 - float64(m)
    j := min(int(math.Hypot(x, y)/float64(m) * bands), bands - 1)
    sum[j] += math.Abs(2 * chance[i] - 1)
    cnt[j]++
}
fmt.Printf("░░m=%2d:", m)
for j := 0; j < bands; j++ {
    if cnt[j] == 0 {
        fmt.Print("░░░░░.░")
    } else {
        fmt.Printf("░%5.2f", sum[j]/float64(cnt[j]))
    }
}
fmt.Println()

```

This code is used in section 43.

46. Index.

- a*: [25](#), [37](#).
Abs: [45](#).
across: [12](#), [13](#), [30](#).
agrees: [19](#), [22](#), [27](#), [29](#).
all: [2](#), [3](#).
along: [30](#).
append: [7](#), [8](#), [13](#), [14](#), [15](#), [36](#).
b: [6](#), [7](#), [8](#), [9](#), [25](#), [37](#), [38](#), [39](#), [42](#), [43](#), [44](#), [45](#).
bad: [34](#), [35](#), [38](#), [42](#).
bands: [45](#).
bdd: [22](#).
better: [25](#).
big: [19](#), [23](#), [28](#), [44](#).
bigStr: [19](#).
black: [40](#).
blue: [38](#), [42](#).
board: [5](#), [6](#).
bool: [4](#), [5](#), [6](#), [10](#), [11](#), [12](#), [13](#), [18](#), [26](#), [31](#), [32](#), [33](#), [36](#), [37](#), [39](#).
bucket: [14](#).
buckets: [14](#), [15](#).
Builder: [9](#), [16](#).
c: [4](#), [7](#), [21](#).
cells: [5](#), [7](#), [8](#), [9](#), [39](#), [44](#), [45](#).
chance: [44](#), [45](#).
Cmp: [20](#), [27](#), [29](#).
cnt: [45](#).
Count: [20](#), [22](#), [23](#), [26](#), [27](#), [29](#), [44](#).
d: [15](#), [17](#), [18](#), [20](#), [22](#), [23](#), [24](#), [25](#), [26](#), [27](#), [29](#), [30](#), [31](#), [32](#), [33](#), [34](#), [35](#), [38](#), [43](#), [44](#).
Dance: [18](#).
den: [28](#).
Diagram: [17](#), [32](#).
diamond: [11](#), [14](#), [15](#), [31](#), [32](#), [33](#).
dnTri: [10](#), [13](#), [15](#).
domino: [5](#), [8](#).
down: [11](#), [13](#), [14](#), [26](#).
Duration: [17](#).
e: [42](#).
Elt: [44](#).
elts: [34](#), [35](#), [36](#), [38](#), [39](#).
f: [23](#).
flag: [2](#).
Float: [23](#), [44](#).
Float64: [23](#), [44](#).
float64: [24](#), [44](#), [45](#).
fmt: [9](#), [16](#), [19](#), [20](#), [21](#), [22](#), [23](#), [24](#), [25](#), [26](#), [27](#), [29](#), [30](#), [31](#), [32](#), [33](#), [34](#), [35](#), [38](#), [43](#), [45](#).
formula: [19](#).
Fprintf: [9](#), [16](#).
got: [19](#).
growth: [24](#).
h: [12](#), [13](#), [14](#), [15](#), [16](#), [26](#), [27](#), [31](#), [32](#), [33](#), [34](#), [35](#), [36](#).
hexagon: [11](#), [12](#), [32](#), [36](#).
hi: [31](#), [32](#), [33](#).
horiz: [5](#), [39](#), [44](#).
Hypot: [45](#).
i: [7](#), [8](#), [9](#), [13](#), [15](#), [16](#), [28](#), [32](#), [35](#), [37](#), [38](#), [45](#).
inS: [4](#), [7](#), [21](#).
inside: [42](#).
Int: [19](#), [28](#).
int: [4](#), [5](#), [6](#), [7](#), [8](#), [10](#), [11](#), [12](#), [15](#), [19](#), [28](#), [29](#), [31](#), [32](#), [33](#), [34](#), [36](#), [37](#), [38](#), [39](#), [40](#), [42](#), [43](#), [45](#).
int64: [28](#).
interlaced: [34](#), [35](#), [36](#).
Ints: [36](#).
Itoa: [32](#).
j: [8](#), [15](#), [28](#), [45](#).
k: [5](#), [8](#), [15](#), [19](#), [28](#), [32](#), [36](#), [39](#), [44](#).
kind: [11](#), [31](#), [33](#), [36](#).
l: [10](#), [11](#), [12](#), [13](#), [15](#), [28](#), [31](#), [36](#).
len: [7](#), [13](#), [14](#), [32](#), [36](#), [37](#), [44](#).
lo: [31](#), [32](#), [33](#).
Lsh: [19](#).
m: [4](#), [5](#), [6](#), [7](#), [10](#), [11](#), [12](#), [13](#), [15](#), [20](#), [21](#), [28](#), [31](#), [36](#), [38](#), [43](#), [45](#).
macMahon: [27](#), [28](#), [29](#).
main: [1](#).
make: [14](#), [32](#), [36](#), [44](#).
math: [23](#), [24](#), [45](#).
MaxWeight: [31](#), [32](#).
min: [21](#), [45](#).
mode: [2](#), [3](#).
more: [42](#).
MRV: [18](#).
mrv: [17](#), [18](#).
Mul: [28](#).
n: [4](#), [5](#), [6](#), [7](#), [10](#), [11](#), [12](#), [13](#), [15](#), [22](#), [23](#), [24](#), [25](#), [27](#), [28](#), [29](#), [30](#), [31](#), [33](#), [42](#).
New: [18](#), [35](#), [38](#).
new: [19](#), [23](#), [44](#).
newBoard: [6](#), [20](#), [22](#), [24](#), [25](#), [38](#), [43](#).
newHexagon: [12](#), [26](#), [27](#), [29](#), [30](#), [31](#), [33](#), [34](#), [35](#).

- NewInt*: 19, 28.
NewPCG: 35, 38.
NewReader: 18.
next: 39, 41, 42.
Nodes: 20, 22, 24, 25, 27, 29, 30.
Now: 18.
num: 5, 6, 7, 8, 11, 12, 13, 15, 28, 42.
o: 9, 16, 31, 32, 33, 36, 39, 44.
ok: 39, 41, 42.
ok1: 8.
ok2: 8.
opts: 5, 8, 9, 11, 14, 16, 32, 36, 39, 44.
p: 5, 8, 9, 11, 16, 39, 40, 44, 45.
Parse: 2.
pick: 32.
Pow: 23.
pow2: 19, 20.
prev: 24, 39, 41, 42.
Print: 45.
printed: 10, 12, 13, 26.
Printf: 20, 21, 22, 23, 24, 25, 26, 27, 29, 30, 31, 33, 34, 35, 38, 45.
Println: 20, 22, 24, 25, 26, 27, 30, 31, 33, 34, 38, 43, 45.
q: 5, 8, 9, 11, 16, 24, 44.
Quo: 28, 44.
Quotient: 44.
r: 4, 7, 20, 22, 23, 24, 27, 29.
rand: 35, 38.
Random: 35, 38.
red: 38, 42.
result: 17, 18.
rnd: 35, 38.
root: 34, 35, 38, 44.
row: 21.
rows: 36.
rowwise: 6, 7.
s: 31, 34, 41, 42.
s32: 23.
sb: 9, 16.
secs: 17.
seen: 34.
segs: 39, 40, 41.
SetInt: 23, 44.
share: 44.
Signatures: 25.
Since: 18.
solve: 18, 20, 22, 24, 25, 26, 27, 29, 30, 31, 33, 34, 35, 38, 43.
Solver: 17.
sort: 36.
span: 31, 32, 33.
spanStr: 31, 32, 33.
Sprintf: 19, 24, 32.
Sqrt: 24.
strconv: 32.
strictlyBetween: 36, 37.
String: 2, 9, 16, 23.
string: 5, 11, 18, 19, 26, 32.
strings: 9, 16, 18.
Subsets: 34.
sum: 45.
sv: 17, 18, 25.
t: 13, 14, 15, 18, 26.
test: 5, 9, 11, 16, 18, 20, 22, 24, 25, 26, 27, 29, 30, 31, 33, 34, 35, 38, 43.
time: 17, 18.
total: 44.
tri: 11, 12, 13, 15.
tris: 11, 13, 14, 16, 26.
twice: 41.
uint: 19.
uint64: 38.
up: 26.
upTri: 10, 13.
v: 36, 37.
w: 21, 32.
want: 19, 22.
white: 40.
WriteByte: 9, 16.
x: 10, 11, 13, 15, 36, 45.
y: 10, 11, 13, 15, 33, 36, 45.
z: 34, 35, 38, 44.
ZDD: 34, 35, 38, 44.
zdd: 1, 17, 18, 32.

- ⟨Add the segments, watching for a clash 41⟩ Used in section 39
- ⟨Check that row r of $S_{m(2m)}$ has $2\min(r+1, 2m-r)$ cells 21⟩ Used in section 20
- ⟨Check the two counts the exercise asks for 23⟩ Used in section 22
- ⟨Count the diamonds of each kind 31⟩ Used in section 3
- ⟨Count the tilings of an Aztec diamond 20⟩ Used in section 3
- ⟨Cross a blue segment and a red one over the domino 40⟩ Used in section 39
- ⟨Do what the mode asks 3⟩ Used in section 1
- ⟨Find the chance that each cell is covered lengthwise 44⟩ Used in section 43
- ⟨Follow the segments and count the paths 42⟩ Used in section 38
- ⟨Functions 4⟩ Used in section 1
- ⟨Lay the segments of one tiling 39⟩ Used in section 38
- ⟨Look for the arctic circle 43⟩ Used in section 3
- ⟨Make the diamonds 14⟩ Used in section 12
- ⟨Make the dominoes 8⟩ Used in section 6
- ⟨Measure the diagram of S_n 22⟩ Used in section 3
- ⟨Measure the diagram of T_n 26⟩ Used in section 3
- ⟨Number the cells 7⟩ Used in section 6
- ⟨Number the cells the other way 24⟩ Used in section 3
- ⟨Number the triangles 13⟩ Used in section 12
- ⟨Offer the three diamonds that cover (x, y) 15⟩ Used in section 14
- ⟨Read the command line 2⟩ Used in section 1
- ⟨Report how far from undecided the cells are, band by band 45⟩ Used in section 43
- ⟨Sample $T_{8,8,4}$ at random and check the interlacing 35⟩ Used in section 34
- ⟨Trace Randall's paths 38⟩ Used in section 3
- ⟨Types 5⟩ Used in section 1
- ⟨Weigh the two branching rules 25⟩ Used in section 3
- ⟨Write the DLX text 9⟩ Used in section 6
- ⟨Write the hexagon's DLX text 16⟩ Used in section 12

Tilings, and the diagrams that hold them

	Section	Page
Introduction	1	1
The shape S_{mn}	4	3
The hexagon T_{lmn}	10	5
Running the engine	17	8
Aztec diamonds	20	9
The size of the diagram	22	10
Which way to number the cells	24	11
Fewest options, or least numbered	25	12
The hexagon's items	26	13
Diamonds by kind, and by row	31	15
Randall's paths	38	18
The arctic circle	43	21
Index	46	23